Assignment # 2

Generative AI **Deadline: November 10, 2026**

In this assignment, you will research, fine-tune, evaluate, and deliver a complete Transformer-based system that matches patient descriptions to individual inclusion and exclusion criteria of clinical trials. Your system must justify its decisions using evidence from the patient record and demonstrate advanced Transformer inference mechanisms, including KV caching, efficient attention, and speculative decoding.

This is an individual research-and-implementation assignment. You must independently investigate research papers, official model documentation, modern techniques, and relevant implementations beyond what has been taught in class. You are expected to test competing design choices, report failures, and defend every major technical decision. Running an existing TrialGPT repository or producing a notebook-only demonstration is not sufficient.

Instructions

- This assignment must be completed individually and submitted through Google Classroom by the given deadline. A grace period of two hours will be allowed after the official deadline, during which submissions will still be accepted for evaluation without any penalty. **Submissions received after the grace period will be not even evaluated and ZERO marks be awarded as per the course's late submission policy**.
- Use any freely available computational environment, including Kaggle, Google Colab, university facilities, or your own hardware. Paid GPUs, paid APIs, or proprietary model services must not be necessary to meet compulsory requirements.
- Use an open-weight, locally executable Transformer. You may choose the model and fine-tuning method, provided the architecture supports the required autoregressive inference work. Explain the computational resources and memory limitations.
- The technical report must follow IEEE research-paper format, prepared in LaTeX. Submit the report PDF and a link to the source project. All experiments and tables must be described and interpreted, rather than merely pasted as screenshots.
- Maintain a GitHub repository containing the source code, environment/dependency specifications, data preparation, training and evaluation scripts, attention and decoding implementations, application source, reproducibility instructions, and results. Host large weights elsewhere and provide a working download method.
- You may use AI assistants and existing libraries, but must acknowledge their use, test reused code, and understand the complete submission. During defense, you may be asked to explain, execute, or change a component and evaluate an unseen case.
- No marks be assigned if you miss your demo.

Mandatory Dataset and Common Evaluation Protocol

All students must use the TrialGPT Criterion Annotations dataset: https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations . The original project is

available at https://github.com/ncbi-nlp/TrialGPT and may be studied as a reference only. No replacement dataset is permitted for the core supervised training and comparison. Public trial metadata may be accessed only to support the application; it must not replace the common labeled benchmark.

The dataset contains patient descriptions, individual inclusion/exclusion criterion text, expert eligibility judgments, and expert-identified evidence sentences. It also contains previous model-generated predictions and explanations; these must never be provided as input features or leaked into evaluation.

Use a patient-grouped split: sort unique patient IDs, shuffle using NumPy default_rng(42), and assign the first 37 patients to training, the next 8 to validation, and the remaining 8 to testing. Keep every annotation belonging to one patient in exactly one split. Save and publish the split manifest (patient IDs and counts). The final test set must remain untouched until final evaluation. Do not tune models, prompts, or thresholds on the test cases.

All groups must evaluate the same criterion-level tasks and use the common metrics specified below. Different model choices are allowed, but comparisons must clearly identify the model and decoding settings. The application may evaluate multiple criteria for a patient, yet the dataset provides criterion-level annotations; any overall patient–trial verdict should be described as an aggregation based on explicit rules, not presented as a separately ground-truth-labeled outcome.

Task 1: Research, Architecture Selection, and Transformer Fine-Tuning

Investigate at least five credible technical or scholarly sources relevant to clinical eligibility reasoning, Transformer architectures, supervised adaptation, and efficient inference. Identify at least two reasonable open-weight decoder model candidates and justify the final choice with parameter counts, licensing, expected memory consumption, architecture, and suitability for the available hardware.

Fine-tune the selected model on the prescribed training split so that it accepts (i) a patient description, (ii) criterion type, and (iii) criterion text, and returns a structured eligibility label and supporting patient-sentence IDs. You may select any suitable fine-tuning method; Low-Rank Adaptation (LoRA) is one optional technique to investigate, not a required method.

Report the data preprocessing, tokenization, output schema, training objective, training configuration, selected hyperparameters, validation performance, and final held-out results. Compare the same model before and after fine-tuning. Truncation must be documented, especially when it could remove relevant patient evidence.

Task 2: Criterion-Level Eligibility and Evidence Verification

Build a reliable prediction and evidence pipeline for both inclusion and exclusion criteria. Preserve the dataset's applicable expert label categories, including insufficient-information and not-applicable outcomes; do not silently collapse them into a binary yes/no decision. For each prediction, return the criterion, predicted category, cited sentence IDs, extracted evidence text, and an explanation based only on the available information.

Evaluate criterion classification using accuracy, macro-F1, per-class precision/recall/F1, and confusion matrices, reported both overall and separately for inclusion versus exclusion criteria. Evaluate evidence sentence retrieval using precision, recall, and F1; specify how records with no annotated evidence are treated. Present representative errors, contradictory cases, and failures due to missing information. Never treat absent evidence automatically as proof of either eligibility or ineligibility.

Task 3: KV-Cached Autoregressive Generation

Implement a decoding path that uses and updates the Transformer's key/value cache across generated tokens. Establish a reference path without cache reuse. Under deterministic greedy decoding, verify token-level agreement for the same inputs and compare prefill time, decoding time, output tokens per second, and peak memory under several input and output lengths.

Using a library switch to enable caching without examining or implementing the cache-aware decoding flow is insufficient. Explain positional handling, masking, cache growth, and possible numerical differences.

Task 4: Efficient Tiled Attention with Online Softmax

Implement a correct reference scaled dot-product attention computation and an attention variant based on tiling and online softmax. Verify output agreement on small controlled examples with causal masking and numerically stable tolerances. Compare speed and memory use where meaningful; explain why an educational PyTorch implementation may not outperform a fused kernel.

Students may investigate supported PyTorch optimized attention backends. No particular CUDA-only third-party kernel is compulsory. The assessment concerns conceptual and implementation correctness on freely accessible hardware, not availability of a specific accelerator architecture.

Task 5: Speculative Decoding

Select an appropriate smaller draft model with tokenization compatible with the target Transformer and implement greedy speculative decoding: candidate-token drafting, target-model verification, accepting matching tokens, handling rejection, and maintaining correct cache state. Compare with ordinary greedy decoding on the same prompts; verify generated token equality and report draft acceptance statistics, overall latency, tokens/second, and memory.

Use existing assisted-decoding libraries for reference testing if desired, but simply calling an existing assisted-generation API is not enough. A speedup is not guaranteed; correct implementation, measured evidence, and explanation of slowdowns are essential.

Task 6: Integrated Browser-Based Application

Deliver one functional application with a standalone browser-accessible interface. A user must be able to supply a patient record, select or enter a trial's eligibility criteria, run the

selected Transformer, inspect each criterion-level result and the exact supporting evidence, and export a readable eligibility screening report. Include a clear notice that this is an educational screening prototype, not a clinical enrollment decision.

The application must expose a meaningful inference comparison or benchmark view for the implemented decoding mechanisms. Training and research may take place in notebooks, but an evaluator must not need to execute code manually in Kaggle, Colab, a notebook, command line, or IDE to use the final application. A browser-accessible locally hosted or deployed application is acceptable; document the startup process clearly.

Technical Report and Submission

Write an IEEE-format technical research paper in LaTeX with: Title and author details; Abstract and keywords; Introduction and problem statement; Related Work; Materials and Methods (dataset, split, selected model, fine-tuning, architecture diagrams, attention and decoding algorithms, and application workflow); Experimental Setup and Metrics; Results and Discussion; Limitations; Conclusion; References. Include training curves, relevant tables and graphs, measured runtimes, error analysis, and screenshots. Each figure and table must be referred to and interpreted in the text.

Submit the report PDF, GitHub URL, reproducible execution instructions, the trained model weights or adapter download link, split manifest, benchmark logs, and a five-to-seven-minute recorded application demonstration with a video link. Include a brief AI-use appendix. During live evaluation, the application may be tested with previously unseen patient descriptions and criteria.

Evaluation Rubric (100 Marks)

The following rubric will be used for assessment. Evidence must be visible in the working application, source code, experiment outputs, and written report. Marks within each category depend on correctness, reproducibility, clarity, and independent understanding.

Code	Criterion	Full-credit evidence / observable standard	Marks
A. Research and architecture (15)			
A1	Literature and critical comparison	At least five credible sources; compare alternatives and their limitations; connect selected techniques to the problem.	5
A2	Model choice and feasibility	Correctly report architecture, parameters, tokenizer, context window, licensing and compute requirements; justify selection.	5
A3	Research-informed technical decisions	Document alternatives considered, rejected options, observed difficulties and evidence behind final design.	5

Code	Criterion	Full-credit evidence / observable standard	Marks
B. Dataset and model adaptation (20)			
B1	Dataset integrity and fixed split	Use only prescribed TrialGPT dataset and 37/8/8 patient-grouped split with seed 42; preserve IDs; provide manifest and checks for patient leakage.	5
B2	Input/output data processing	Correctly prepare patient text, criterion type and text, eligibility labels and sentence-ID evidence; exclude GPT-4 predictions from input.	5
B3	Fine-tuning implementation	Provide executable fine-tuning, checkpoints/adapters, reproducible configuration, training curves/logs and a validated final checkpoint.	5
B4	Before/after model assessment	Evaluate identical validation/test examples before and after fine-tuning; document decoding settings and discuss differences.	5
C. Eligibility and evidence quality (20)			
C1	Eligibility label correctness	Handle inclusion/exclusion label sets, unknown and not-applicable cases; report held-out accuracy and macro-F1.	5
C2	Detailed classification metrics	Provide class precision/recall/F1, confusion matrices and separate inclusion/exclusion analysis using common test split.	5
C3	Evidence sentence localization	Return original sentence IDs; report micro/macro evidence precision, recall and F1 with handling of empty gold/predicted sets.	5
C4	Traceable explanations and failure analysis	Each prediction links to source evidence; analyze at least eight distinct errors, including unsupported or missing-information cases.	5
D. KV-cached inference (15)			
D1	Cache-aware decoder implementation	Use model forward passes with explicit cache reuse/update, causal masking and positional handling; not only use_cache=True.	5
D2	Correctness tests	Demonstrate greedy-token equivalence with uncached reference across at least three input/output lengths; investigate mismatches.	5
D3	Benchmark quality	Measure prefill, decode, tokens/s and peak memory with same model and inputs, warm-up and synchronized repeated timing.	5
E. Efficient attention (15)			

Code	Criterion	Full-credit evidence / observable standard	Marks
E1	Reference attention	Implement scaled dot-product attention with correct scaling, masking and numerically stable softmax.	5
E2	Tiled online-softmax attention	Implement blocked attention with running maxima/normalizers and correct accumulation, without simply invoking a fused backend.	5
E3	Numerical and performance validation	Test multiple shapes and causal masks; report maximum/mean numerical error and honest runtime/memory comparisons.	5
F. Speculative decoding (15)			
F1	Draft/target design	Identify compatible tokenizer and draft/target pair; justify memory and generation settings.	5
F2	Working greedy speculative algorithm	Implement drafting, target verification, accepted-prefix handling, rejection correction, and cache/state synchronization.	5
F3	Correctness and efficiency analysis	Test token equivalence to greedy target; report acceptance rate, end-to-end latency, throughput and speedup or slowdown.	5
G. Integrated application (20)			
G1	Independent browser application	Accessible without running code from IDE/notebook; clear documented startup, model loading, input validation and error handling.	5
G2	Complete screening workflow	Accept unseen clinical note and trial criteria; show each decision, evidence, missing information and overall preliminary outcome.	5
G3	Inference and experiment visibility	Expose inference modes and meaningful result/performance comparison, not static charts or screenshots.	5
G4	Usability and report export	Readable interface, persistent or downloadable assessment report with provenance; demonstrate recovery from invalid inputs.	5
H. Research report and defense (15)			
H1	IEEE paper and scientific interpretation	IEEE LaTeX paper with complete methods, metrics, graphs/tables, discussion, limitations, and properly cited literature.	5
H2	Reproducible source and	Working GitHub repository, dependency/setup instructions, model link, fixed split, logs and 5–7-minute demonstration	5

Code	Criterion	Full-credit evidence / observable standard	Marks
	demonstration	video.	
H3	Individual technical defense	Accurately explain model, cache, attention, decoding and results; reproduce or modify a chosen component on request.	5
TOTAL			**135**

Performance-Level Descriptors

For each subcriterion, marks are awarded in proportion to demonstrated evidence: Full (90–100% of subcriterion marks): fully correct, tested, reproducible and clearly justified. Strong (70–89%): functional with minor omissions or weak analysis. Developing (40–69%): partial implementation, limited tests, or material deficiencies. Minimal (1–39%): isolated, weak, or non-reproducible evidence. Zero: missing, nonfunctional, plagiarized, or unsupported work. Round to the nearest 0.5 mark where appropriate.

Important grading rules: (i) No marks for results copied from TrialGPT without student implementation and verification. (ii) Accuracy and evidence scores require the fixed held-out test cases. (iii) Claiming use of a cache or attention kernel without validating the implemented mechanism does not satisfy the respective criterion. (iv) A speculative decoder may receive full implementation marks even if it is slower, provided its correctness and measured performance are documented. (v) An IDE-only/notebook-only demonstration cannot receive full application marks. (vi) The system is a research screening prototype, not a medical enrollment decision tool.